



GaliciaWear

Viste a beiramar, teje futuro

Marketplace de Moda Sostenible Gallega

Aplicación multiplataforma: Android · Web · Escritorio · API REST

Memoria de Proyecto Fin de Ciclo

CFGS Desarrollo de Aplicaciones Multiplataforma (DAM)

Alumno Pablo Suárez Varela

Curso académico 2025/2026

Centro CPR Plurilingüe Karbo

Tutor Ramón Carrasco Borrego

Fecha de entrega Junio de 2026

Documento elaborado para la convocatoria ordinaria · TFG DAM 2024-26

Abstract

GaliciaWear is a cross-platform e-commerce marketplace for sustainable Galician fashion that connects local artisan designers with eco-conscious consumers. Its scope covers four coordinated software products built on a single shared back end: a native **Android** app (customers), a **React** web storefront with a designer dashboard, a **JavaFX** desktop admin panel, and a **Node.js REST API** backed by **MySQL** and **MongoDB**. The methodology was iterative and phase-based, under Git version control with a GitHub Actions CI pipeline running on Linux. Key results include role-based authentication with JWT access and refresh tokens, a full sustainable catalogue with verified certificates (GOTS, OEKO-TEX, GRS), multi-designer orders with ecological shipping, real-time notifications and chat over Socket.IO, an administration panel with bulk XML/JSON import/export, an automated backup script, and a test suite of more than 160 cases across the four clients. The project delivers a production-ready system that exercises the complete DAM curriculum — from OOP, ORM and relational/NoSQL databases to REST APIs, real-time communications, security and multiplatform UX.

Resumen (es). GaliciaWear es un marketplace multiplataforma de moda sostenible gallega que conecta a diseñadores artesanales locales con consumidores concienciados. Integra una app **Android** nativa, una **web** React (tienda + panel de diseñador), un panel de administración de escritorio en **JavaFX** y una **API REST** en Node.js sobre **MySQL** y **MongoDB**. Incorpora autenticación por roles con JWT, catálogo con certificados de sostenibilidad verificados, pedidos multidiseñador con envío ecológico, notificaciones y chat en tiempo real (Socket.IO), panel de administración con importación/exportación XML/JSON, copia de seguridad automatizada y una batería de más de 160 pruebas. El resultado es un sistema completo que recorre todos los contenidos del ciclo DAM.

Índice

1	Portada	1
2	Abstract / Resumen	2
3	Índice	3
4	Justificación del proyecto	8
4.1	Motivación	8
4.2	Relevancia	8
4.3	Viabilidad	8
4.4	Concreción	8
5	Introducción	9
5.1	Contexto y estado del arte	9
5.2	Revisión técnica de partida	9
5.3	Planteamiento	9
6	Desarrollo técnico	10
6.1	Arquitectura y metodología	10
6.1.1	Metodología de trabajo	10
6.1.2	Arquitectura del sistema	10
6.1.3	Diagramas UML	11
6.2	Tecnologías y entorno de desarrollo	14
6.2.1	Lenguajes y paradigmas	14
6.2.2	Entorno, <i>build</i> y control de versiones	15
6.2.3	Compatibilidad con Linux	15
6.3	Gestión de datos y persistencia	15
6.3.1	Base de datos relacional (MySQL + Prisma)	16
6.3.2	ORM en dos plataformas	17
6.3.3	Base de datos NoSQL (MongoDB)	17

6.3.4	Importación y exportación XML/JSON	17
6.3.5	Copia de seguridad automatizada	17
6.4	Desarrollo de aplicaciones y comunicaciones	18
6.4.1	Aplicación móvil Android (Java)	18
6.4.2	Aplicación web (React)	20
6.4.3	Aplicación de escritorio (JavaFX)	22
6.4.4	API REST (<i>backend</i> Node.js)	23
6.4.5	Comunicación entre procesos y tiempo real	24
6.5	Interfaz, UX y criterios psicológicos	24
6.5.1	Gestión de usuarios, roles y acceso	25
6.6	Seguridad, <i>testing</i> y documentación	25
6.6.1	Seguridad	25
6.6.2	Estrategia de pruebas	26
6.6.3	Documentación	26
6.6.4	Trazabilidad de contenidos DAM	26
7	Análisis del uso de inteligencia artificial	28
7.1	Herramientas de IA empleadas	28
7.2	Ámbitos de aplicación	28
7.3	Beneficios y eficiencia obtenida	28
7.4	Limitaciones, riesgos y validación humana	28
7.5	Ética, propiedad intelectual y transparencia	29
7.6	Impacto en el aprendizaje y competencias DAM	29
7.7	Declaración de uso responsable de IA	29
8	Conclusiones	29
9	IPE I — Empresa, RRHH y marco laboral	31
9.1	Prevención de Riesgos Laborales (PRL)	31
9.2	Contratos laborales	31
9.3	Recursos humanos y organización	31

10 IPE II — Forma jurídica y viabilidad	33
10.1 Forma jurídica y justificación	33
10.1.1 Viabilidad económica	33
10.1.2 Fuentes de financiación	33
10.2 Business Model Canvas	33
11 Digitalización aplicada a los sectores productivos	34
11.1 Procesos internos digitalizados	34
11.2 Impacto en eficiencia y competitividad	35
A International IT Overview (English)	36

Índice de figuras

Índice de figuras

1	Arquitectura del sistema: tres clientes consumen una API REST común sobre persistencia políglota (MySQL + MongoDB), con tiempo real (Socket.IO) y <i>push</i> (FCM). Empaquetado y despliegue con Docker sobre Linux.	11
2	Casos de uso del actor Cliente (app Android): acceso, exploración del catálogo con filtros de sostenibilidad, compra con envío ecológico, seguimiento, chat y reseñas.	12
3	Casos de uso de los actores Diseñador (web, izquierda) y Administrador (escritorio, derecha). El alta de diseñador requiere validación del administrador.	13
4	Diagrama de secuencia del flujo de compra: la creación del pedido se realiza en una transacción MySQL atómica (bloqueo de <i>stock</i> con <code>SELECT ... FOR UPDATE</code>) y la notificación al diseñador es <i>fire-and-forget</i> vía Socket.IO + FCM.	14
5	Modelo entidad-relación (MySQL). Las entidades se agrupan por color: usuarios (azul), catálogo (ocre), comercio (azul marino) y social/certificados (verde).	16
6	App Android: pantalla de autenticación (izquierda) y catálogo con tarjetas de producto, distancia de origen y material (derecha).	18
7	App Android: ficha de producto con material, certificados de sostenibilidad y selección de talla/color (izquierda); cesta con control de cantidad, total y tramitación del pedido (derecha).	19
8	App Android: perfil de cliente con avatar, rol (CLIENTE) y accesos a edición y soporte. . .	20
9	Web: página de inicio del <i>storefront</i> (« <i>Viste a beiramar, teje futuro</i> ») con la propuesta de valor y los distintivos de proximidad, certificación, envío ecológico y artesanía. . . .	21
10	Web: panel del diseñador (Liñares Moda) con indicadores de prendas publicadas, ventas y pedidos por aceptar, y accesos a la gestión del catálogo.	21
11	Web: chat en tiempo real entre cliente y diseñador (Socket.IO), con estado de conexión e historial de mensajes.	22
12	Escritorio (JavaFX): <i>dashboard</i> de administración con métricas globales (usuarios, clientes, diseñadores, ingresos) y reparto de pedidos por estado.	23

Índice de tablas

Índice de cuadros

1	Resumen del <i>stack</i> tecnológico por capa.	15
---	--	----

2	Trazabilidad de contenidos DAM, ubicación y evidencia.	27
3	Business Model Canvas (orientación sostenible) de GaliciaWear.	34

4 Justificación del proyecto

4.1 Motivación

El comercio de moda sostenible y de proximidad carece en Galicia de plataformas digitales integradas que pongan en contacto a las pequeñas marcas y diseñadores artesanales con un público comprador concienciado. Las soluciones genéricas disponibles en el mercado — *marketplaces* como Etsy, plataformas de creación de tiendas como Shopify o aplicaciones de segunda mano como Vinted — no contemplan los elementos que definen este nicho: la verificación de certificados de sostenibilidad, la logística ecológica, la trazabilidad del producto o la identidad cultural gallega. El proyecto nace, por tanto, de una necesidad real detectada: dar visibilidad digital y un canal de venta moderno a un tejido productivo local que hoy opera de forma fragmentada, principalmente a través de redes sociales generalistas y ventas presenciales.

4.2 Relevancia

GaliciaWear persigue un impacto en tres planos. Para los **diseñadores**, supone un escaparate profesional y herramientas de gestión (catálogo, pedidos, mensajería y métricas) sin necesidad de conocimientos técnicos. Para los **consumidores**, ofrece acceso a moda ética con información verificable sobre materiales, certificaciones y kilómetros de origen, reduciendo el *greenwashing*. Para el **sector textil gallego**, contribuye a su digitalización y a la fijación de talento y actividad económica en el territorio. En el plano académico, el proyecto integra de forma práctica la totalidad de los módulos del ciclo de Desarrollo de Aplicaciones Multiplataforma.

4.3 Viabilidad

La viabilidad técnica se sustenta en un *stack* íntegramente *open source* (Node.js, React, Android, JavaFX, MySQL, MongoDB, Docker) y en los conocimientos previos del alumno en desarrollo móvil, *backend* y web. El desarrollo se planificó en fases acotadas a lo largo del curso, con control de versiones y automatización (CI) que permitieron sostener cuatro aplicaciones con un equipo de una sola persona. Los recursos de despliegue son contenibles: el sistema funciona sobre un único servidor con servicios en contenedores Linux.

4.4 Concreción

El interés del nicho no es meramente teórico: la moda sostenible es uno de los segmentos de mayor crecimiento del sector textil europeo, impulsado por la creciente exigencia regulatoria y de los consumidores en materia de trazabilidad y economía circular. Galicia, con una larga tradición textil y un número creciente de pequeñas marcas de autor, presenta una oferta atomizada y con escasa presencia digital consolidada. GaliciaWear se concreta en una propuesta verificable, no en una declaración de

intenciones: cada prenda del catálogo enlaza con sus certificados (GOTS, OEKO-TEX, GRS, entre otros), su material principal y su distancia de origen, datos que se modelan explícitamente en la base de datos y se muestran al comprador en las tres aplicaciones cliente.

5 Introducción

5.1 Contexto y estado del arte

El proyecto se inscribe en la intersección de tres tendencias: el comercio electrónico, la moda sostenible y la transformación digital del pequeño comercio. En el ámbito del *e-commerce*, el patrón arquitectónico dominante es el de un *backend* de servicios que expone una API y varios clientes que la consumen; este enfoque, frente a las aplicaciones monolíticas, favorece la reutilización, la escalabilidad y el desarrollo multiplataforma. En el ámbito de la sostenibilidad, las plataformas existentes adolecen de una limitación común: tratan la sostenibilidad como una etiqueta de *marketing* y no como un dato estructurado y verificable.

Del análisis de las alternativas se concluye que **Etsy** resuelve el *marketplace* artesanal pero es global y no verifica sostenibilidad; **Vinted** se centra en la segunda mano sin relación con el diseñador; **Shopify** habilita tiendas individuales pero no construye comunidad ni proximidad; y las grandes plataformas de *fast fashion* operan en el extremo opuesto del modelo que aquí se propone. Ninguna combina, de forma simultánea, sostenibilidad certificada, logística ecológica e identidad regional.

5.2 Revisión técnica de partida

El diseño se apoya en conceptos consolidados de la ingeniería del *software*: arquitectura *cliente-servidor* y separación en capas; el estilo de diseño *REST* para la API; el patrón *MVVM* en el cliente Android y *MVC* en el cliente de escritorio; el mapeo objeto-relacional (ORM) para la persistencia; y la comunicación en tiempo real mediante *WebSockets*. Estos fundamentos, tratados a lo largo del ciclo, se aplican aquí de forma integrada sobre un caso de uso real.

5.3 Planteamiento

GaliciaWear se plantea como una solución integral y multiplataforma. Sobre una única API *REST* se construyen tres clientes con públicos y propósitos diferenciados —consumidor (Android), diseñador (web) y administración (escritorio)— que comparten modelo de datos, seguridad y lógica de negocio. La diferenciación respecto al estado del arte reside en tratar la sostenibilidad como información de primera clase del dominio y en ofrecer una experiencia de usuario moderna y coherente en las tres plataformas. Los apartados siguientes detallan la arquitectura, las tecnologías, la gestión de datos, el desarrollo de cada aplicación, las decisiones de interfaz, la seguridad y las pruebas.

6 Desarrollo técnico

Este capítulo constituye el núcleo de la memoria. Se organiza en seis subapartados que recorren la arquitectura y metodología, las tecnologías, la gestión de datos, el desarrollo de las aplicaciones y comunicaciones, la interfaz y la experiencia de usuario, y la seguridad, pruebas y documentación. Por concisión, se incluyen únicamente fragmentos de código destacables (de quince líneas o menos), comentados y acompañados de su explicación. El sistema suma en torno a **26.800 líneas de código** propias repartidas entre los cuatro componentes.

6.1 Arquitectura y metodología

6.1.1 Metodología de trabajo

El desarrollo siguió un enfoque **iterativo e incremental por fases**, gestionado con un tablero *Kanban* personal apoyado en *GitHub Issues* como *backlog*. El control de versiones se realizó con **Git** sobre **GitHub**, empleando ramas por funcionalidad, *pull requests* y mensajes de *commit* semánticos; cada integración en las ramas principales dispara la *integración continua*. El proyecto se estructuró en nueve fases (0–8):

- **Fase 0–1.** Inicialización del monorepo, Docker, CI y modelado UML/ER.
- **Fase 2.** *Backend*: autenticación, usuarios, catálogo, comercio, comunicaciones, *Socket.IO*, Swagger y datos de prueba (*seed*).
- **Fase 3.** Persistencia avanzada: MongoDB, copia de seguridad e importación/exportación XML/JSON.
- **Fase 4.** Aplicación Android (pantallas, FCM, notificaciones).
- **Fase 5.** Panel de administración JavaFX con *dashboard* en tiempo real y empaquetado con *j package*.
- **Fase 6.** Aplicación web completa (*storefront* + panel de diseñador).
- **Fase 7–8.** Seguridad transversal, cobertura de pruebas, CI completa y documentación.

6.1.2 Arquitectura del sistema

La arquitectura es **cliente-servidor organizada en capas**. Una API REST centraliza toda la lógica de negocio y la seguridad, y es consumida por tres clientes independientes —Android, web y escritorio— que comparten el mismo modelo de datos. Sobre esta base se superpone una capa de **tiempo real** (*Socket.IO* sobre *WebSockets*) para chat y notificaciones, y un canal de **notificaciones push** (Firebase Cloud Messaging) para el segundo plano. Internamente, el *backend* sigue el patrón **Controlador** → **Servicio** → **Repositorio**, lo que separa el transporte HTTP, la lógica de negocio y el acceso a datos. La persistencia es políglota: **MySQL** para el núcleo transaccional y **MongoDB** para datos flexibles (*logs*, multimedia de reseñas y recomendaciones).

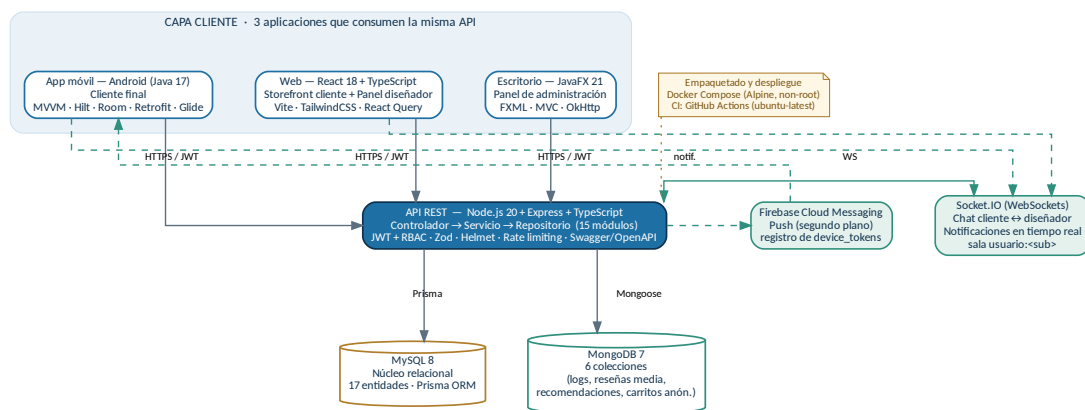


Figura 1. Arquitectura del sistema: tres clientes consumen una API REST común sobre persistencia polígloa (MySQL + MongoDB), con tiempo real (Socket.IO) y *push* (FCM). Empaquetado y despliegue con Docker sobre Linux.

6.1.3 Diagramas UML

El modelado se documentó con diez diagramas (PlantUML y Mermaid) en docs/uml/: ER, clases del dominio, despliegue, dos de secuencia y tres de casos de uso. La diagrama de despliegue se corresponde con la arquitectura ya mostrada en la Figura anterior. A continuación se resumen los casos de uso por actor (Cliente, Diseñador y Administrador), que reflejan los tres clientes del sistema.

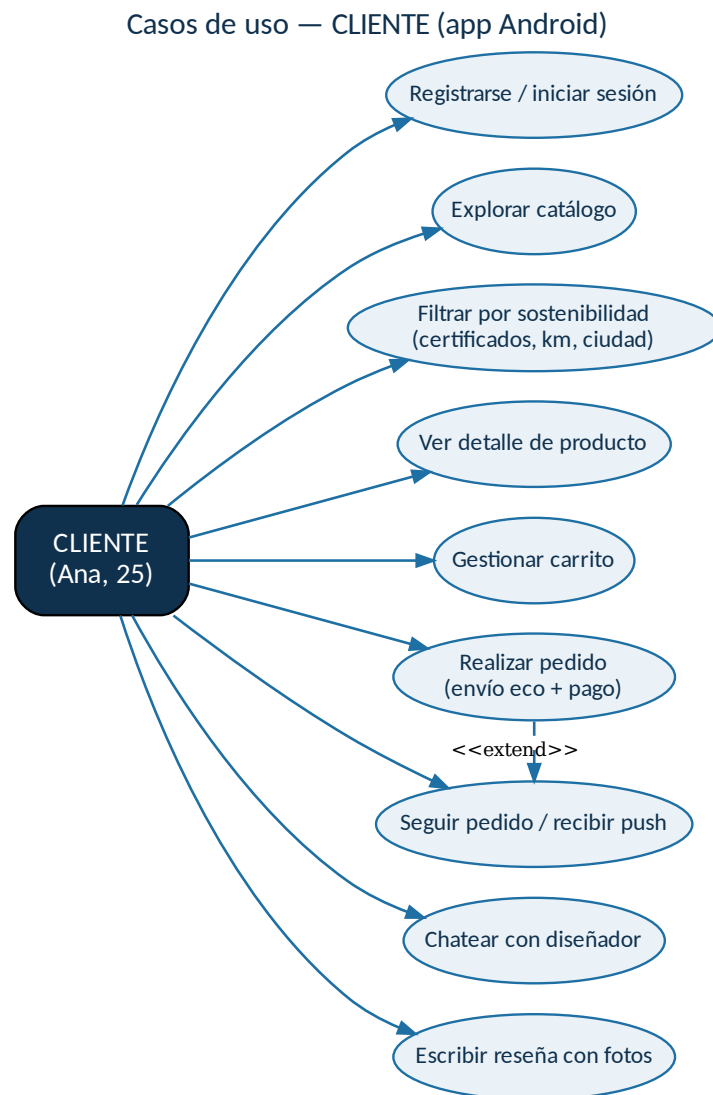
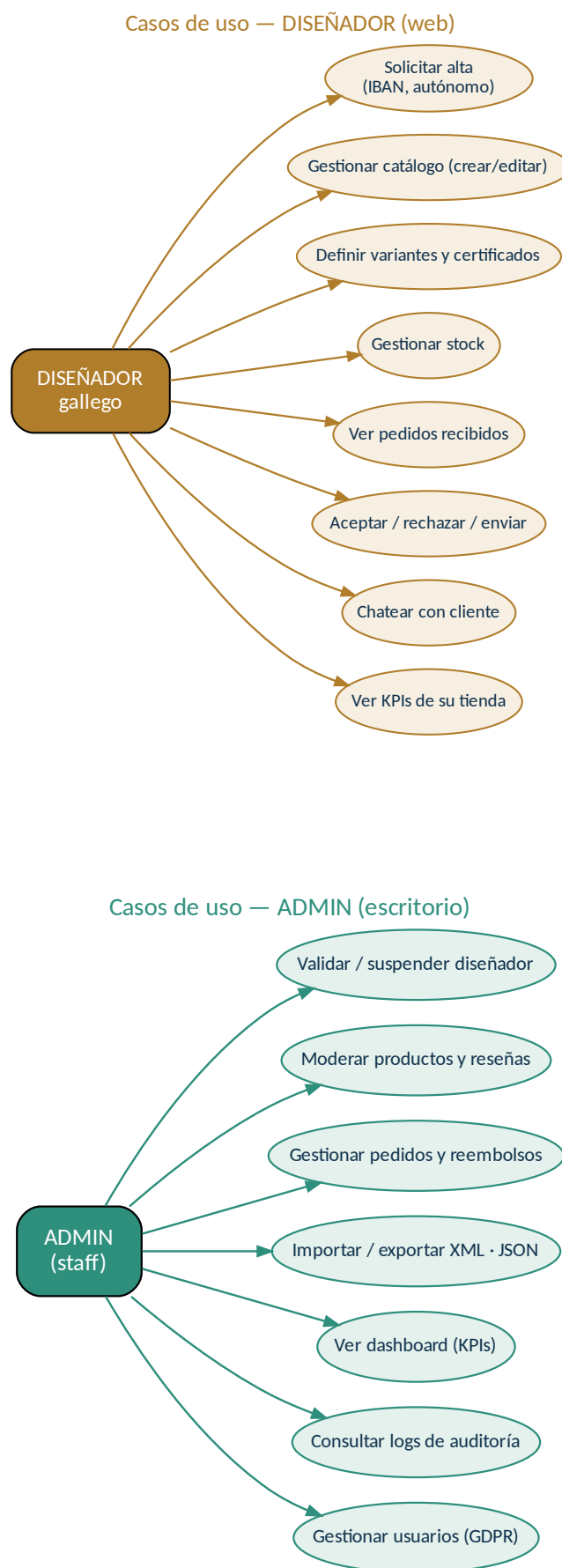
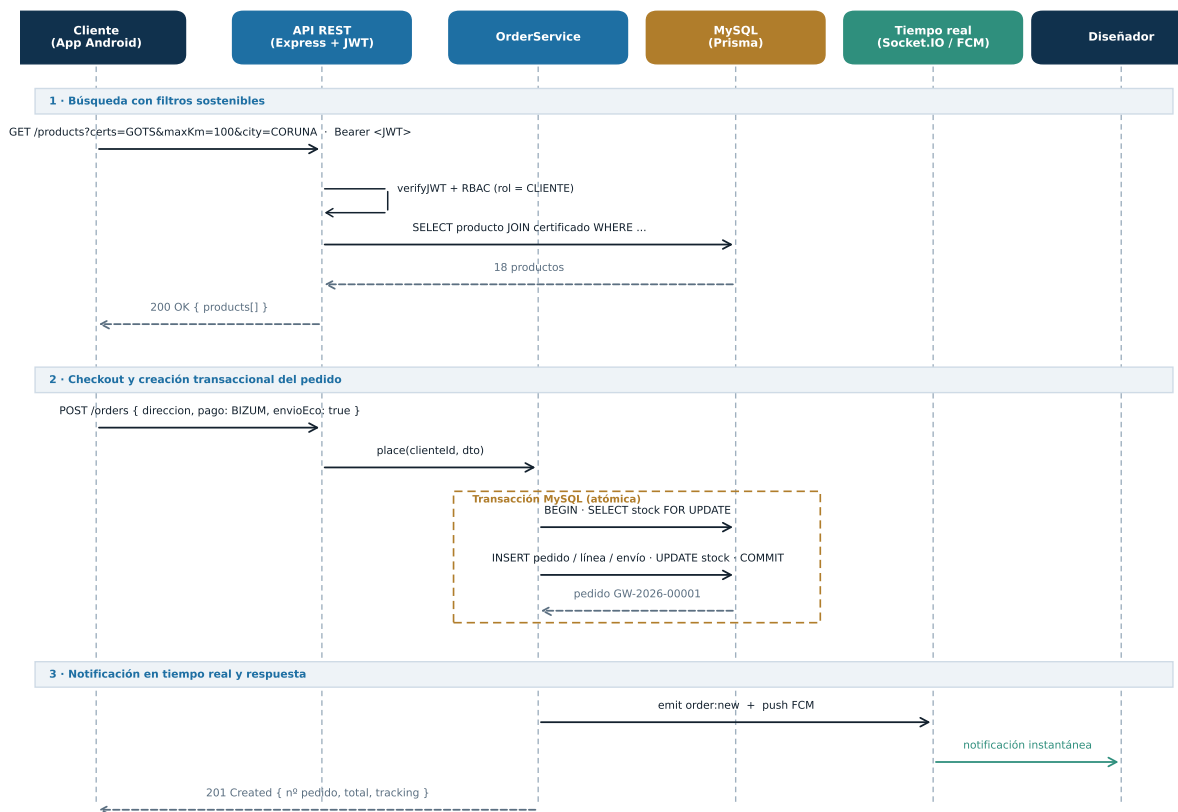


Figura 2. Casos de uso del actor Cliente (app Android): acceso, exploración del catálogo con filtros de sostenibilidad, compra con envío ecológico, seguimiento, chat y reseñas.



El flujo más representativo del sistema es la **compra**, que recorre desde la búsqueda con filtros sostenibles hasta el registro transaccional del pedido y la notificación en tiempo real al diseñador. Su diagrama de secuencia se muestra en la Figura 4.



Flujo end-to-end típico: 8-10 s. El cliente no espera por la notificación del diseñador (fire-and-forget).

Figura 4. Diagrama de secuencia del flujo de compra: la creación del pedido se realiza en una transacción MySQL atómica (bloqueo de stock con `SELECT ... FOR UPDATE`) y la notificación al diseñador es *fire-and-forget* vía Socket.IO + FCM.

6.2 Tecnologías y entorno de desarrollo

6.2.1 Lenguajes y paradigmas

El proyecto combina varios lenguajes según la plataforma: **Java 17** (Android y escritorio), **TypeScript/JavaScript** (*backend* y web), **SQL** (migraciones gestionadas por el ORM) y **FXML** (definición declarativa de las vistas JavaFX). Se aplican distintos paradigmas de forma deliberada: **programación orientada a objetos** en Java (herencia, interfaces y polimorfismo en las entidades de dominio y los repositorios), **programación reactiva/funcional** en React (*hooks*, composición de componentes y estado declarativo), el patrón **MVVM** en Android y **MVC** en JavaFX, y una organización **modular orientada a características** (*feature-based*) en el *backend*. Por convención del proyecto, los identificadores se escriben en castellano (véase `CONVENCIONES.md`).

Tabla 1. Resumen del *stack* tecnológico por capa.

Componente	Tecnologías principales	LOC aprox.
App móvil (Android)	Java 17 · MVVM · Hilt · Room · Retrofit · Glide · FCM	9.570
Web (cliente + diseñador)	React 18 · TypeScript · Vite · TailwindCSS · React Query	8.440
API REST (<i>backend</i>)	Node.js 20 · Express · TypeScript · Prisma · Socket.IO	6.700
Escritorio (admin)	JavaFX 21 · Maven · FXML · MVC · OkHttp	2.090
Datos: MySQL 8 (Prisma ORM) + MongoDB 7 (Mongoose)		—
Infra: Docker Compose · GitHub Actions · Linux (Alpine/Ubuntu)		—

6.2.2 Entorno, *build* y control de versiones

Los entornos de desarrollo empleados fueron **Android Studio** (Android), **IntelliJ IDEA** y **Visual Studio Code** (*backend*, web y FXML). Cada componente usa su sistema de construcción nativo: **Gradle** (Android), **Maven** (escritorio), **npm/Vite** (web) y el compilador **tsc** (*backend*). La integración continua se define en `.github/workflows/ci.yml` y ejecuta *lint* y pruebas de *backend* y web en cada *push* o *pull request*.

6.2.3 Compatibilidad con Linux

El cumplimiento del requisito de compatibilidad con Linux es transversal. Toda la orquestación se realiza con **Docker Compose** (imágenes *Alpine*, usuario no *root*); los *scripts* de operación (`start-dev.sh`, `backup.sh`, `restore.sh`) son *shell* con `set -euo pipefail` y entrecomillan las rutas para tolerar espacios; el panel JavaFX se empaqueta con `jpackage` a `.deb` y `.AppImage`; y la CI se ejecuta sobre `ubuntu-latest`. El Código 1 muestra un extracto del flujo de CI.

Código 1. Extracto de `ci.yml`: job de *backend* en Ubuntu.

```

1 jobs:
2   backend:
3     name: Backend (Node 20)
4     runs-on: ubuntu-latest
5     defaults: { run: { working-directory: backend } }
6     steps:
7       - uses: actions/checkout@v4
8       - uses: actions/setup-node@v4
9         with: { node-version: '20', cache: 'npm' }
10      - run: npm install --no-audit --no-fund
11      - run: npm run lint
12      - run: npm test

```

6.3 Gestión de datos y persistencia

6.3.1 Base de datos relacional (MySQL + Prisma)

El núcleo del sistema se almacena en una base de datos relacional **MySQL 8**, gestionada mediante el ORM **Prisma**. Se eligió MySQL por disponer el centro de un servidor MySQL sin limitaciones, y porque soporta de forma nativa los tipos empleados (ENUM, Decimal, Json y Text). El esquema (database/schema.prisma, 424 líneas) define **17 entidades** y 9 enumerados, agrupables en cuatro dominios: *usuarios* (Usuario, Cliente, Diseñador, Direccion, TokenRefresco), *catálogo* (Producto, Variante, ImagenProducto, CertificadoSostenibilidad y la tabla puente CertificadoDeProducto), *comercio* (Carrito, ItemCarrito, Pedido, LineaPedido, Envio) y *social* (Resena, Mensaje).

El diseño incorpora decisiones relevantes: identidad por UUID, borrado lógico (*soft-delete*) mediante *fecha_eliminacion*, separación de Usuario y sus perfiles Cliente/Diseñador según el rol, y un pedido que se descompone en una LineaPedido por diseñador (lo que permite pedidos multimarca). La convención mapea identificadores Prisma en *camelCase* a columnas en *snake_case* mediante *@map*. La Figura 5 muestra el modelo entidad-relación completo.

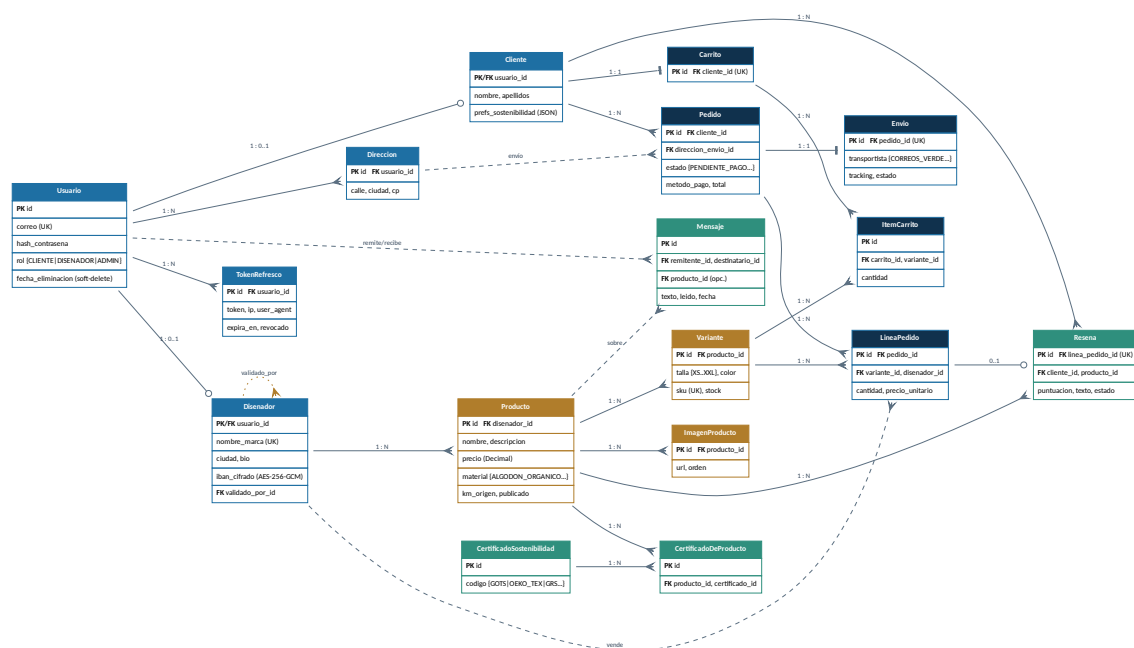


Figura 5. Modelo entidad-relación (MySQL). Las entidades se agrupan por color: usuarios (azul), catálogo (ocre), comercio (azul marino) y social/certificados (verde).

Código 2. Entidad Variante en Prisma (talla, color, SKU único y stock).

```
1 model Variante {
2   id String @id @default(uuid())
3   productoId String @map("producto_id")
4   talla TallaPrenda
5   color String
6   sku String @unique
7   stock Int @default(0)
```



```

8   producto Producto @relation(fields: [productoId],
9       references: [id], onDelete: Cascade)
10  @map("variante")
11  }

```

6.3.2 ORM en dos plataformas

El mapeo objeto-relacional aparece en dos lugares del proyecto: **Prisma** en el *backend* (consultas tipadas, migraciones versionadas y transacciones) y **Room** en la app Android, que persiste localmente una caché de catálogo y la sesión, habilitando un uso fluido con conectividad intermitente.

6.3.3 Base de datos NoSQL (MongoDB)

Para los datos que no encajan en un esquema rígido se emplea **MongoDB 7** (vía Mongoose), con **seis colecciones**: `device_tokens` (tokens FCM por dispositivo), `review_media` (multimedia de reseñas), `activity_logs` y `notification_logs` (auditoría y notificaciones), `recommendations` (preferencias eco personalizadas) y `anonymous_carts` (cestas de visitantes no registrados). El caso de uso paradigmático es `device_tokens`: un usuario puede tener un número variable de dispositivos, cada uno con metadatos distintos, lo que se modela con naturalidad como documento flexible y se consulta sin coste de *joins*.

6.3.4 Importación y exportación XML/JSON

La API expone y consume datos estructurados en JSON (catálogo, pedidos, usuarios). El panel de administración añade **importación y exportación masiva** de productos y pedidos en **XML y JSON**: el analizador `fast-xml-parser` convierte entre XML y objetos, y las operaciones costosas se delegan a *worker threads* para no bloquear el bucle de eventos. Esta funcionalidad cubre el requisito de interoperabilidad de datos de la rúbrica.

6.3.5 Copia de seguridad automatizada

El *script* `scripts/backup.sh` realiza un volcado consistente de MySQL (`mysqldump --single-transaction`) y de MongoDB (`mongodump`), los comprime en un `.tar.gz` con marca temporal y aplica una política de retención de 30 días. Se acompaña de `restore.sh` para la restauración. La automatización recomendada es una tarea cron diaria (`0 3 * * *`). El Código 3 reproduce el núcleo del *script*.

Código 3. Núcleo de `backup.sh`: volcado de MySQL y MongoDB, compresión y retención.

```

1 # Volcado consistente sin bloquear tablas InnoDB
2 MYSQL_PWD="$MYSQL_PASS" mysqldump --host="$MYSQL_HOST" --port="$MYSQL_PORT" \
3   --user="$MYSQL_USER" --single-transaction --routines --triggers --events \
4   "$MYSQL_DB" > "${DIR_TMP}/mysql.sql"

```

```
5 mongodump --uri="$MONGO_URI" --out="${DIR_TMP}/mongo" --quiet
6 tar -czf "${BACKUPS_DIR}/${NOMBRE}.tar.gz" -C "$BACKUPS_DIR" "tmp_${FECHA}"
7 # Retencion: elimina copias con mas de 30 dias
8 find "$BACKUPS_DIR" -name "galiciawear_*.tar.gz" -mtime +30 -print -delete
```

6.4 Desarrollo de aplicaciones y comunicaciones

6.4.1 Aplicación móvil Android (Java)

El cliente del consumidor es una aplicación **Android nativa en Java**, con arquitectura **MVVM** e inyección de dependencias mediante **Hilt**. La capa de datos separa el origen local (**Room**) del remoto (**Retrofit**) tras una capa de repositorios; la interfaz se construye con **ViewBinding**, **LiveData** y **ViewModel**. La app comprende **16 actividades y 9 fragmentos** (39 *layouts*): *splash*, autenticación, catálogo, detalle de producto, carrito, *checkout*, pedidos, chat, conversaciones, notificaciones, perfil y edición, además de las vistas propias del rol diseñador. La comunicación en tiempo real se realiza con el cliente **Socket.IO** y las notificaciones *push* con **Firebase Cloud Messaging**.

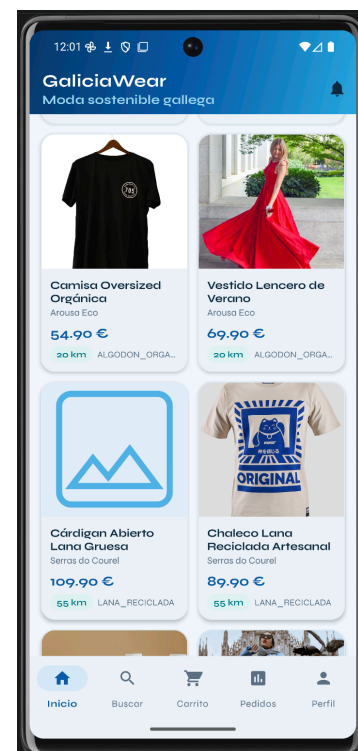


Figura 6. App Android: pantalla de autenticación (izquierda) y catálogo con tarjetas de producto, distancia de origen y material (derecha).



Figura 7. App Android: ficha de producto con material, certificados de sostenibilidad y selección de talla/color (izquierda); cesta con control de cantidad, total y tramitación del pedido (derecha).



Figura 8. App Android: perfil de cliente con avatar, rol (CLIENTE) y accesos a edición y soporte.

6.4.2 Aplicación web (React)

La web, construida con **React 18 + TypeScript + Vite**, cumple un doble propósito sobre **20 páginas**: un **storefront** público para clientes (catálogo, filtros, carrito, *checkout* e historial) y un **panel de diseñador** (gestión de productos, pedidos, mensajes y métricas). Emplea **React Query** para la gestión del estado de servidor y la caché, **Socket.IO** para chat y notificaciones, **TailwindCSS** para un diseño *responsive* y **React Router 6** para la navegación SPA.

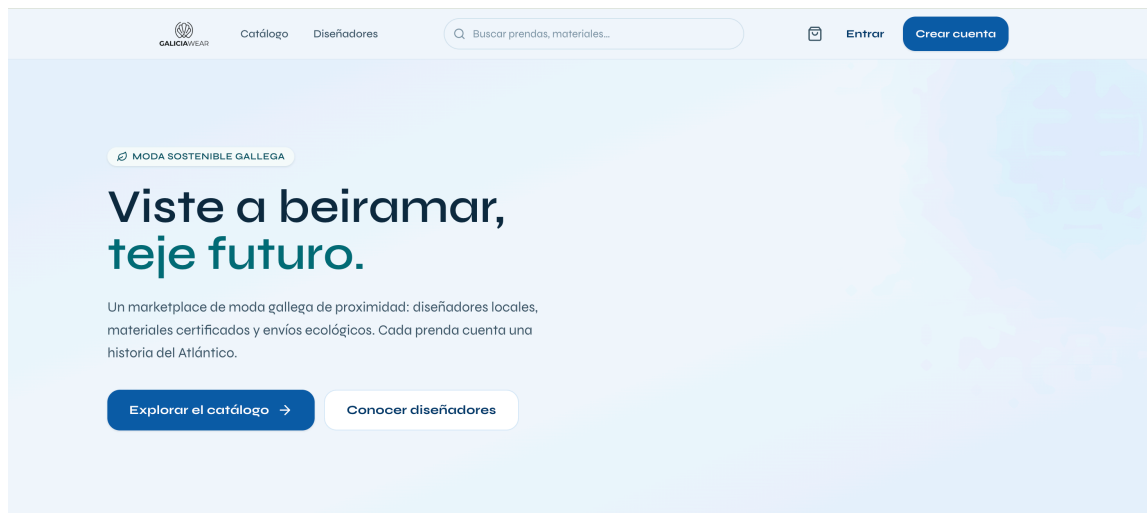


Figura 9. Web: página de inicio del storefront («Viste a beiramar, teje futuro») con la propuesta de valor y los distintivos de proximidad, certificación, envío ecológico y artesanía.

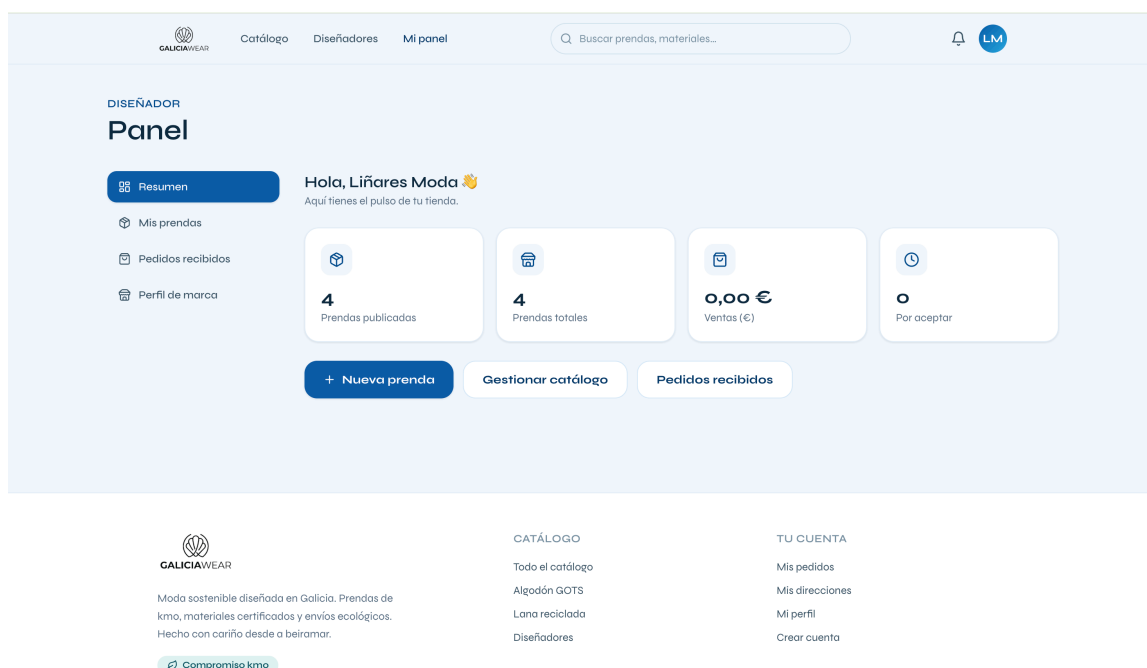


Figura 10. Web: panel del diseñador (Liñares Moda) con indicadores de prendas publicadas, ventas y pedidos por aceptar, y accesos a la gestión del catálogo.

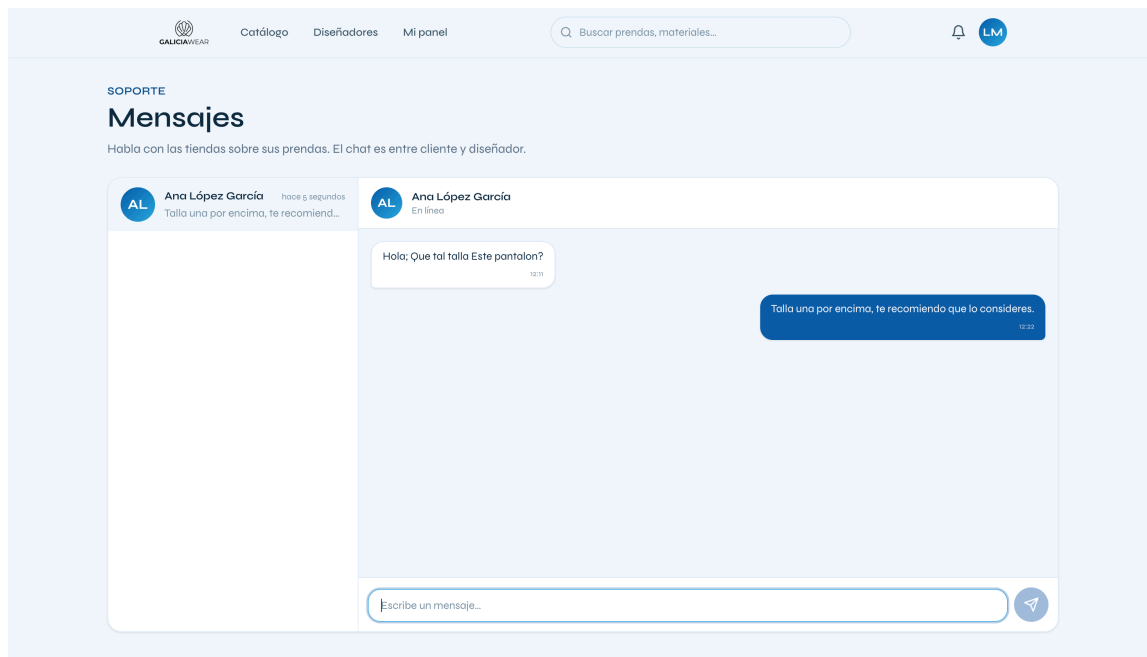


Figura 11. Web: chat en tiempo real entre cliente y diseñador (Socket.IO), con estado de conexión e historial de mensajes.

6.4.3 Aplicación de escritorio (JavaFX)

El panel de administración es una aplicación de escritorio **JavaFX 21** (Maven), empaquetable con **jpackage**. Sigue el patrón **MVC** con vistas declaradas en **FXML** y consume la API mediante un cliente **OkHttp** (**CienteHttp**). Dispone de **ocho vistas** —acceso, panel principal, *dashboard*, diseñadores, productos, pedidos, logs de auditoría e importación/exportación— con diálogos modales para las operaciones CRUD y un *dashboard* que se actualiza en tiempo real. Supera holgadamente el requisito de la rúbrica de una interfaz gráfica de escritorio (mínimo Swing).

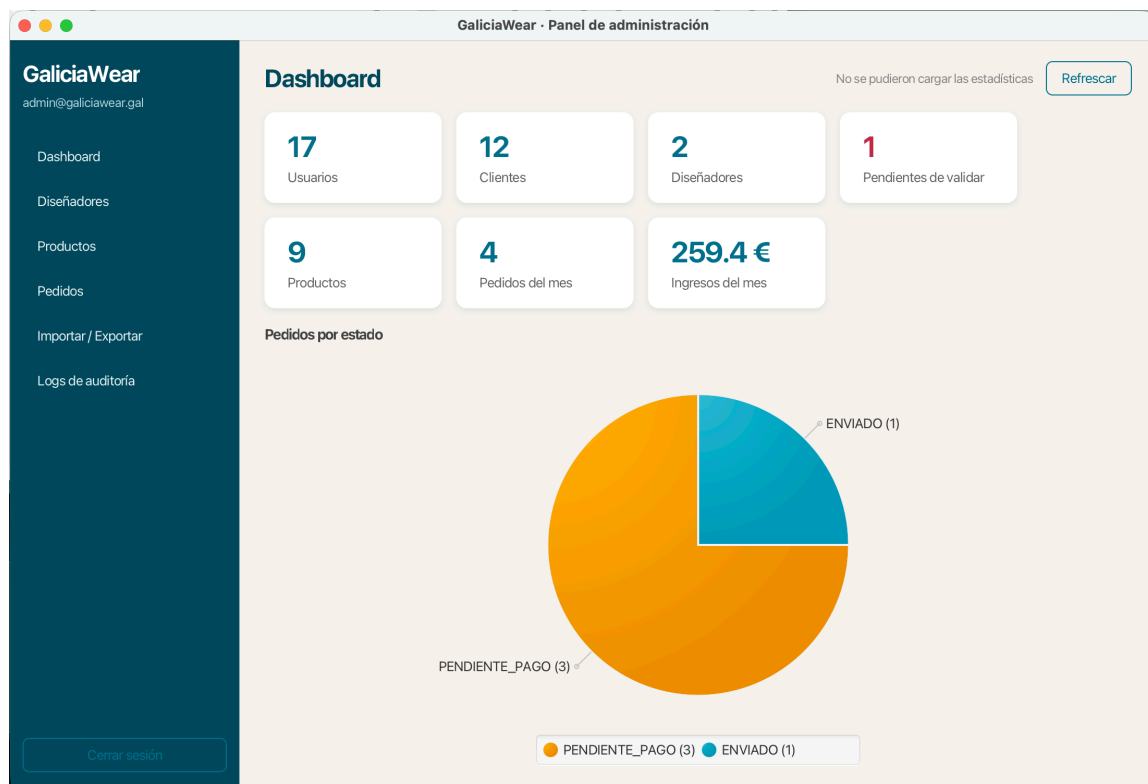


Figura 12. Escritorio (JavaFX): *dashboard* de administración con métricas globales (usuarios, clientes, diseñadores, ingresos) y reparto de pedidos por estado.

6.4.4 API REST (*backend Node.js*)

El *backend* es una **API REST** en **Node.js + Express + TypeScript** organizada en **15 módulos** por característica: autenticación, usuarios, diseñadores, direcciones, certificados, productos, variantes, carrito, pedidos, envíos, chat, notificaciones, imágenes, mongo y admin, con cerca de **80 manejadores de ruta**. Cada módulo agrupa controlador, servicio y validación. La autenticación JWT y el control de acceso por roles se aplican como *middleware*; la validación de entradas se realiza con **Zod**; y la API se documenta con **Swagger/OpenAPI** (disponible en `/api/docs`). El Código 4 muestra el *middleware* de control de acceso por roles.

Código 4. Middleware RBAC: `requerirRol` y atajos semánticos por rol.

```
1 export function requerirRol(...rolesPermitidos: Rol[]) {
2   return (peticion, _respuesta, siguiente) => {
3     if (!peticion.usuario)
4       return siguiente(new ErrorNoAutenticado('Necesita autenticarse'));
5     if (!rolesPermitidos.includes(peticion.usuario.rol))
6       return siguiente(new ErrorAccesoDenegado('Rol no autorizado'));
7     siguiente();
8   };
9 }
10 export const soloAdmin = requerirRol(Rol.ADMIN);
11 export const diseñadorOAdmin = requerirRol(Rol.DISENADOR, Rol.ADMIN);
```

6.4.5 Comunicación entre procesos y tiempo real

La capa de tiempo real se implementa con **Socket.IO** sobre *WebSockets*. Al conectarse, cada usuario se une a una **sala personal** usuario:<sub>, de modo que el servicio de notificaciones puede emitir el evento *nueva_notificacion* a todos sus dispositivos sin recurrir a FCM cuando la aplicación está activa. El chat cliente-diseñador usa una **sala determinista por pareja** (salaPar), lo que garantiza que ambos comparten canal con independencia de quién inicie la conversación. Los tres clientes se conectan al mismo servidor de *sockets*; **FCM** cubre el *push* en segundo plano mediante el registro de *tokens* de dispositivo (su envío queda como punto de extensión productivo). En cuanto a la concurrencia, el *backend* aprovecha el **bucle de eventos** de Node y *worker threads* para tareas intensivas (importación/exportación), mientras que Android ejecuta las llamadas de red en hilos en segundo plano para no bloquear la interfaz.

Código 5. Socket.IO: sala personal por usuario y difusión del mensaje de chat a la sala de la pareja.

```
1 // Sala personal: llega a cualquier dispositivo del usuario, sin FCM
2 void socket.join(`usuario:${usuario.sub}`);
3 // Chat: io.to(sala) incluye al emisor (eco inmediato en su propia UI)
4 io.to(salaPar(usuario.sub, peerId)).emit('nuevo_mensaje', mensaje);
```

6.5 Interfaz, UX y criterios psicológicos

El diseño de las interfaces se rige por los criterios de usabilidad y psicología cognitiva tratados en el ciclo, aplicados de forma consistente en las tres aplicaciones cliente.

- **Carga cognitiva.** Los flujos complejos se dividen en pasos discretos (el *checkout* se descompone en cesta, dirección/envío y pago) y la información de la ficha de producto se agrupa por relevancia: datos, variantes, certificados y, por último, la acción de compra.
- **Feedback inmediato.** Toda acción produce una respuesta visible (*snackbars* y *toasts*, indicadores de progreso) y las acciones destructivas piden confirmación previa.
- **Consistencia.** Las tres aplicaciones comparten sistema de color, tipografía e iconografía, y sitúan las acciones primarias en posiciones equivalentes.
- **Ley de Fitts.** Los botones de acción principal (añadir al carrito, pagar) son de gran tamaño y se anclan en zonas de fácil alcance (parte inferior en móvil), como se aprecia en las Figuras de la cesta y el detalle.
- **Ley de Hick.** Las opciones de filtrado se agrupan y se ocultan por defecto para no saturar la decisión del usuario, mostrando un número reducido de controles visibles.
- **Accesibilidad.** Se emplean descripciones de contenido (*contentDescription*) en las imágenes de Android, contraste conforme a WCAG AA y tamaños de fuente mínimos legibles.

6.5.1 Gestión de usuarios, roles y acceso

El sistema define tres roles —**CLIENTE**, **DISEÑADOR** y **ADMIN**— con vistas y permisos diferenciados. El control de acceso es doble: en el *frontend* se condicionan menús y rutas según el rol, y en el *backend* se verifica de forma autoritativa mediante el *middleware* de autorización (Código 4). Esta separación evita confiar la seguridad al cliente y mantiene una única fuente de verdad sobre los permisos. Cada rol dispone, además, de su propia aplicación: el cliente en Android, el diseñador en la web y el administrador en el panel de escritorio.

6.6 Seguridad, testing y documentación

6.6.1 Seguridad

La seguridad se aborda de forma transversal:

- **Contraseñas.** Se almacenan con **bcrypt** (sal por contraseña), nunca en claro.
- **Sesiones.** Autenticación con **JWT**: *token* de acceso de corta vida (15 min) y *token* de refresco (7 días) almacenado en la tabla `TokenRefresco` junto a *IP* y *User-Agent*, con revocación activa al cerrar sesión.
- **Datos sensibles.** El IBAN del diseñador se cifra con **AES-256-GCM** (IV de 96 bits, etiqueta de autenticación) antes de persistirse; la clave reside en una variable de entorno, nunca en la base de datos (Código 6).
- **Validación de entradas.** Todos los *endpoints* validan tipo, longitud y formato con **Zod**.
- **Cabeceras y red.** **Helmet** (XSS, CSP, HSTS), **CORS** con lista blanca de orígenes y *rate limiting* en las rutas de autenticación.
- **Despliegue.** Contenedores Docker con usuario no *root* sobre imagen *Alpine* mínima.

Código 6. Cifrado AES-256-GCM del IBAN: IV aleatorio y etiqueta de autenticación (detecta manipulación).

```
1 export function cifrarTexto(texto: string): string {
2   const iv = crypto.randomBytes(12); // 96 bits, recomendado para GCM
3   const cifrador = crypto.createCipheriv('aes-256-gcm', obtenerClave(), iv);
4   const cifrado = Buffer.concat([cifrador.update(texto, 'utf8'),
5                                 cifrador.final()]);
6   const tag = cifrador.getAuthTag();
7   return `${iv.toString('hex')}:${tag.toString('hex')}:${cifrado.toString('hex')}`;
8 }
```

6.6.2 Estrategia de pruebas

El proyecto incorpora una batería de pruebas en los cuatro componentes, con **más de 160 casos repartidos en 33 ficheros** de *test*:

- **Backend: Jest** con pruebas unitarias de servicios y de integración de rutas (mediante *supertest* sobre una base de datos de prueba).
- **Web: Vitest + React Testing Library** (componentes, contextos y llamadas a la API).
- **Android: JUnit 5 y Espresso** (pruebas de interfaz instrumentadas).
- **Escritorio: TestFX** (pruebas de la interfaz JavaFX).

Las pruebas se ejecutan automáticamente en la integración continua en cada cambio.

6.6.3 Documentación

La documentación generada comprende la especificación **Swagger/OpenAPI** de la API, los diez diagramas **UML** de docs/uml/, la guía de estilo de código (CONVENCIONES.md), las instrucciones de instalación y despliegue (README.md) y las cuentas de prueba (TESTING_ACCOUNTS.md), además de la presente memoria.

6.6.4 Trazabilidad de contenidos DAM

La Tabla 2 relaciona cada contenido del ciclo con su ubicación en la memoria y la evidencia que lo respalda, para facilitar la labor de evaluación.

Tabla 2. Trazabilidad de contenidos DAM, ubicación y evidencia.

Contenido DAM	Apartado	Evidencia
BBDD relacional + ORM	6.3	<code>schema.prisma</code> , ER (Fig. 5)
BBDD NoSQL	6.3	6 colecciones MongoDB
XML/JSON import/export	6.3	Panel admin, <code>fast-xml-parser</code>
<i>Script</i> de copia de seguridad	6.3	<code>backup.sh</code> (Cód. 3)
OOP y programación	6.2, 6.4	Java (Android/Desktop), TS (<i>backend</i>)
Interfaz gráfica (mín. Swing)	6.4	JavaFX (escritorio)
Aplicación móvil	6.4	App Android nativa
Aplicación web	6.4	React <i>storefront</i> + panel
API REST	6.4	15 módulos, Swagger
WebSockets e hilos	6.4	Socket.IO (Cód. 5), <i>workers</i>
Comunicación entre procesos	6.4	Socket.IO + FCM
Notificaciones en tiempo real	6.4	<code>nueva_notificacion</code> , FCM
Seguridad básica y roles	6.5, 6.6	JWT, bcrypt, AES, RBAC
Multiplataforma	6.4	Android + Web + Escritorio
Compatibilidad Linux	6.2	Docker <i>Alpine</i> , CI Ubuntu
Testing básico	6.6	+160 casos, 33 ficheros
UML y casos de uso	6.1	<code>docs/uml/</code> (10 diagramas)
Criterios psicológicos UX	6.5	Fitts, Hick, carga cognitiva
Control de versiones (Git)	6.2	GitHub + CI

7 Análisis del uso de inteligencia artificial

Nota académica. El uso de inteligencia artificial no se penaliza; se valora su aplicación consciente, documentada y ética. Este capítulo describe con transparencia cómo se empleó la IA como herramienta de apoyo y cómo se validó cada resultado.

7.1 Herramientas de IA empleadas

Se utilizaron dos herramientas. **Claude** (Anthropic), a través de la interfaz de asistencia al desarrollo, fue la herramienta principal, con un uso prácticamente diario para generación de código, depuración, revisión de arquitectura, redacción técnica y planificación de fases. **GitHub Copilot** se usó de forma puntual como autocompletado en el editor para reducir el *boilerplate* repetitivo (adaptadores, DTOs).

7.2 Ámbitos de aplicación

La IA se empleó como apoyo en tareas concretas: sugerencia de fragmentos iniciales de código, ayuda puntual en la depuración de errores y en la redacción de documentación técnica. El desarrollo de la lógica de negocio, la integración de los módulos y las decisiones de diseño se realizaron y validaron manualmente.

7.3 Beneficios y eficiencia obtenida

El principal beneficio fue la reducción del tiempo dedicado a tareas repetitivas y de baja carga cognitiva (estimada en torno a un 40% en el *scaffolding* de nuevos módulos) y la aceleración de la curva de aprendizaje de tecnologías nuevas, como Prisma o Hilt. Un ejemplo concreto: el sistema de *tokens* de refresco con revocación activa (registro de IP y *User-Agent*) se prototipó en una sesión, generando la IA el esquema y el *middleware* iniciales que después se validaron con pruebas de integración.

7.4 Limitaciones, riesgos y validación humana

Se detectaron alucinaciones puntuales, como la sugerencia de versiones de dependencias incompatibles (por ejemplo, una versión de Hilt no compatible con el *plugin* de Gradle), corregidas tras consultar la documentación oficial. Todo el código relacionado con la seguridad (JWT, bcrypt, AES) se revisó manualmente línea a línea antes de integrarse. Como norma, cada salida de la IA se ejecutó localmente, se sometió a las pruebas y se contrastó con la documentación oficial de cada biblioteca. Las decisiones de arquitectura principales —estructura de módulos, gestión de roles y diseño del esquema de datos— se tomaron de forma autónoma y manual.

7.5 Ética, propiedad intelectual y transparencia

No se introdujeron datos reales de usuarios ni credenciales en las herramientas de IA: solo código fuente y descripciones técnicas abstractas. El código generado se consideró un punto de partida, nunca el entregable final. El uso se ajusta a la normativa del centro sobre herramientas generativas: declarado, supervisado y validado.

7.6 Impacto en el aprendizaje y competencias DAM

La IA complementó, sin sustituir, el aprendizaje de los módulos del ciclo. Los conceptos de ORM, sockets, seguridad y testing se asimilaron implementándolos, empleando la IA para resolver bloqueos concretos y no para eludir el proceso. Entre las competencias reforzadas destacan la gestión de *prompts*, el pensamiento crítico sobre el código generado, la depuración guiada y la visión de arquitectura de sistemas.

7.7 Declaración de uso responsable de IA

«Declaro que he utilizado herramientas de inteligencia artificial (Claude y GitHub Copilot) como apoyo en la generación de código inicial, la depuración y la documentación técnica, manteniendo en todo momento la autoría intelectual, la revisión crítica y la validación técnica de los resultados. El código, la arquitectura, las decisiones de diseño y la integración de los módulos DAM han sido supervisados, adaptados y probados por mí, garantizando el cumplimiento de los objetivos académicos del ciclo y la normativa del centro respecto al uso de IA.»

Lugar y fecha: A Coruña, junio de 2026

Firma del alumno: _____
Pablo Suárez Varela

8 Conclusiones

Objetivos frente a logros. El objetivo de construir un sistema multiplataforma completo y cohesionado se ha cumplido: GaliciaWear integra cuatro componentes —app Android, web React, panel de escritorio JavaFX y API REST— sobre una persistencia políglota, con autenticación segura, notificaciones y chat en tiempo real, panel de administración, copia de seguridad automatizada, *pipeline* de integración continua y una batería de más de 160 pruebas. El alcance planteado al inicio se ha materializado en un producto funcional.

Dificultades y soluciones. Durante el desarrollo surgieron retos técnicos que se resolvieron de forma razonada: la sincronización del *token* de refresco entre Android y el *backend* se abordó con un interceptor de Retrofit; la subida de imágenes de producto se resolvió con almacenamiento gestionado

en el servidor; la gestión de salas de Socket.IO con varios roles se simplificó con la sala personal usuario: <sub> y la sala determinista por pareja; y la configuración de Hilt con varios módulos se ordenó con los componentes de inyección adecuados.

Valoración personal. Este proyecto me ha permitido integrar en la práctica todos los módulos del ciclo en un producto real. He reforzado especialmente la comprensión de las arquitecturas cliente-servidor, la importancia de la seguridad desde el diseño y el valor del *testing* continuo y la automatización. Trabajar sobre cuatro plataformas con un único *backend* compartido me ha enseñado a valorar la coherencia del modelo de datos y de los contratos de la API como columna vertebral del sistema.

Líneas de mejora. Como evolución futura se contemplan: la integración de una pasarela de pago real (Stripe o Redsys); el envío *push* productivo de FCM; un motor de recomendaciones basado en el historial de compra; la internacionalización (i18n) para el mercado europeo; y el inicio de sesión social mediante OAuth2/SSO.

9 IPE I — Empresa, RRHH y marco laboral

9.1 Prevención de Riesgos Laborales (PRL)

El puesto de desarrollador conlleva riesgos principalmente ergonómicos y psicosociales que deben prevenirse:

- **Ergonomía.** El trabajo prolongado ante la pantalla exige medidas como situar el monitor a la altura de los ojos, usar silla ergonómica y reposamuñecas, y disponer un soporte para el dispositivo móvil de pruebas.
- **Fatiga visual.** Frente a jornadas largas se aplica la regla 20-20-20 (cada 20 minutos, mirar 20 segundos a 20 pies de distancia), filtros de luz azul y ajuste del brillo al entorno.
- **Estrés y riesgo psicosocial.** Los plazos y la depuración prolongada se gestionan con la técnica Pomodoro (ciclos de 25/5 minutos), descansos activos y comunicación fluida con el tutor.
- **Seguridad eléctrica.** La carga simultánea de varios dispositivos se canaliza con regletas con protección frente a sobretensiones, evitando sobrecargar las tomas.
- **Seguridad informática.** Contraseñas únicas gestionadas con un gestor, verificación en dos pasos en GitHub y servicios en la nube, cifrado del disco del equipo y copias de seguridad periódicas.

9.2 Contratos laborales

Para incorporar perfiles DAM a GaliciaWear en su fase inicial se consideran tres modalidades. El **contrato de formación en alternancia / prácticas formativas** resulta idóneo para titulados recién egresados, con jornada reducida durante el primer año. El **contrato indefinido a tiempo parcial** se reserva para el desarrollador fundador o *lead* una vez la empresa genera ingresos estables. El **contrato temporal por circunstancias de la producción** cubre los picos de trabajo (lanzamientos o campañas estacionales). La elección prioriza el contrato formativo para el primer perfil incorporado, por su encaje con un titulado DAM y su menor coste inicial. En todos los casos se respetan los derechos básicos: 30 días naturales de vacaciones, jornada de referencia de 40 horas semanales (o la reducida que corresponda) y retribución conforme al convenio del sector TIC en Galicia (en torno a 18.000–22.000 €/año para un perfil *junior*).

9.3 Recursos humanos y organización

La previsión de RRHH para escalar el proyecto en los dos primeros años contempla un equipo reducido y polivalente:

- **Backend Developer** (×1): mantenimiento de la API, integraciones de pago y escalabilidad.
- **Android Developer** (×1): nuevas funcionalidades y futura versión iOS.
- **Frontend/Web Developer** (×1): mejoras de UX y panel de analíticas.

-
- **QA Engineer** (×0,5, externo): pruebas manuales y automatizadas.
 - **DevOps/SRE** (×0,5, externo): infraestructura, monitorización y CI/CD.
 - **UX Designer** (×0,5, externo): investigación de usuarios y *wireframes*.
 - **Product Manager / CEO** (×1): estrategia y relación con los diseñadores.

La organización es plana en la fase inicial, con un método de trabajo **Scrum** quincenal y el *backlog* gestionado en GitHub Issues, lo que mantiene la coordinación con una estructura ligera.

10 IPE II — Forma jurídica y viabilidad

10.1 Forma jurídica y justificación

Se compararon las formas jurídicas más habituales para una iniciativa de este tipo. El **autónomo** ofrece mínima burocracia y fiscalidad simple, pero con responsabilidad ilimitada, lo que lo hace inadecuado cuando hay socios o riesgo patrimonial. La **Sociedad Limitada (SL)** requiere un capital mínimo de 3.000 €, limita la responsabilidad al capital aportado, proyecta una imagen profesional y es flexible para incorporar socios. La **SLNE** es una variante simplificada de la SL, menos flexible, y la **cooperativa** aporta ventajas fiscales y carácter democrático a costa de mayor burocracia.

Se elige la **Sociedad Limitada** por cuatro razones: la previsible entrada de socios (diseñadores), la necesidad de una imagen profesional B2B para atraer marcas, la protección del patrimonio personal mediante responsabilidad limitada y la escalabilidad futura (posibles rondas de inversión).

10.1.1 Viabilidad económica

La **inversión inicial** estimada asciende a **8.500 €**, desglosada en servidores e infraestructura *cloud* (1.200 €), dominio y certificados (150 €), constitución de la SL (300 €), *marketing* de lanzamiento (2.000 €), herramientas de desarrollo (500 €) y una reserva operativa (4.350 €). Los **costes fijos mensuales** se estiman en torno a 215 € (servidores 120 €, dominio y correo 15 €, herramientas SaaS 80 €). El **modelo de ingresos** se basa en una comisión del 8% sobre cada venta del *marketplace*: con una previsión conservadora de 50 pedidos/mes y un ticket medio de 60 €, se obtienen unos 2.400 €/mes brutos, situando el punto de equilibrio en torno al tercer o cuarto mes de actividad.

10.1.2 Fuentes de financiación

Se identifican tres vías concretas y adecuadas al perfil del proyecto: un **préstamo ICO Emprendedores** (financiación a tipo fijo para autónomos y pymes de nueva creación); las **subvenciones de la Xunta de Galicia** a startups tecnológicas (programas de apoyo al emprendimiento gallego); y una campaña de **crowdfunding** en una plataforma cívica española (Goteo), coherente con el carácter local y sostenible del proyecto y útil además como validación temprana de mercado.

10.2 Business Model Canvas

Dado el carácter sostenible del proyecto, se adopta un **Sustainable Business Model Canvas** que, además de los nueve bloques clásicos, contempla la dimensión social y ambiental. La Tabla 3 resume el lienzo.

Tabla 3. Business Model Canvas (orientación sostenible) de GaliciaWear.

Segmentos de clientes	Consumidores concienciados (25–45 años) que valoran la moda ética y local; diseñadores y artesanos textiles gallegos que buscan canal de venta digital.
Propuesta de valor	Único <i>marketplace</i> que combina diseño artesanal gallego, certificaciones de sostenibilidad verificadas (GOTS, OEKO-TEX, GRS), logística ecológica y trazabilidad del producto.
Canales	App Android (principal), web <i>responsive</i> , redes sociales (Instagram, TikTok), <i>email marketing</i> y colaboraciones con asociaciones de diseñadores.
Relaciones con clientes	Autoservicio en la app/web, chat directo con el diseñador, notificaciones personalizadas y programa de fidelización por compra sostenible.
Fuentes de ingresos	Comisión del 8% por venta; suscripción <i>premium</i> del diseñador; publicidad contextual (fase 2).
Recursos clave	Plataforma tecnológica (código fuente), base de diseñadores verificados, reputación de marca sostenible y equipo técnico.
Actividades clave	Desarrollo y mantenimiento de la plataforma, <i>onboarding</i> de diseñadores, verificación de certificados, <i>marketing</i> digital y atención al cliente.
Aliados clave	Diseñadores gallegos (proveedores/socios), Correos (logística verde), Xunta de Galicia (subvenciones) y asociaciones textiles gallegas.
Estructura de costes	Infraestructura <i>cloud</i> (fijo), equipo de desarrollo (principal partida), <i>marketing</i> (variable) y verificación de certificados (variable).
Dimensión social/ambiental	Impulso al tejido textil local y al empleo de proximidad; reducción de la huella mediante envíos ecológicos y fomento de materiales certificados y economía circular.

11 Digitalización aplicada a los sectores productivos

11.1 Procesos internos digitalizados

La digitalización no solo define el producto, sino también la gestión y la estrategia del proyecto. Los procesos de negocio que se gestionan digitalmente son:

- **Pedidos y logística.** Flujo digital de extremo a extremo (pedido → pago → asignación al diseñador → envío → entrega) con trazabilidad en tiempo real en la aplicación.
- **Onboarding de diseñadores.** Registro, subida de *portfolio* y validación de certificados de sostenibilidad gestionados desde el panel de administración.
- **Comunicación cliente-diseñador.** El chat *in-app* sustituye llamadas y correos, mejora el tiempo de respuesta y genera un historial indexable.
- **Analítica de negocio.** El *dashboard* de administración ofrece métricas de ventas, productos más vendidos y estados de pedido, habilitando decisiones basadas en datos.
- **Notificaciones y engagement.** El sistema de notificaciones *push* e *in-app* fomenta la recurrencia sin intervención manual del equipo.

- **Copia de seguridad y CI/CD.** La copia de seguridad diaria desatendida y la integración continua (*lint*, pruebas y *build* en cada cambio) eliminan tareas manuales propensas a error y reducen el tiempo de detección de fallos de días a minutos.

11.2 Impacto en eficiencia y competitividad

La digitalización permite a GaliciaWear operar con un equipo mínimo, algo inviable con procesos manuales: un diseñador artesano sin conocimientos técnicos puede gestionar su tienda desde el móvil. Además, la trazabilidad sostenible digitalizada constituye un diferenciador competitivo real frente a los *marketplaces* genéricos, pues el comprador puede consultar en tiempo real los certificados verificados del producto que adquiere.

A International IT Overview (English)

1. Project Introduction. GaliciaWear is a cross-platform marketplace for sustainable Galician fashion, developed as a DAM final project (2025/26) by Pablo Suárez Varela. It connects local Galician designers with eco-conscious consumers through a native Android app, a React web storefront with a designer dashboard, a JavaFX admin panel, and a Node.js REST API.

2. Problem & Solution. No digital platform is specifically designed for sustainable artisan fashion from Galicia. Generic marketplaces (Etsy, Shopify) lack sustainability-certificate verification, ecological logistics, and regional identity. GaliciaWear solves this by combining verified sustainability badges (GOTS, OEKO-TEX, GRS), ecological shipping (Correos Verde), and real-time designer-to-customer communication.

3. Main Features. JWT authentication with refresh-token rotation and active revocation; role-based access control (Cliente, Diseñador, Admin); a full catalogue with sustainability certificates and size/colour variants; shopping cart and multi-designer orders; real-time notifications and chat over Socket.IO; a 15-module REST API with Zod validation and Swagger docs; automated daily backups (mysqldump + mongodump); and XML/JSON import/export.

4. Technologies Used. Languages: Java 17, TypeScript, JavaScript, SQL. Frameworks: Android SDK, React 18, JavaFX 21, Express. Libraries: Hilt, Room, Retrofit, Prisma, Socket.IO, Zod, Helmet, bcrypt. Databases: MySQL 8 and MongoDB 7. Tooling: Docker, GitHub Actions, Vite, Maven, Gradle.

5. User Interface (UI/UX). Cognitive-load reduction through multi-step checkout and information grouping; Fitts's Law applied to large, bottom-anchored primary actions on mobile; Hick's Law respected by limiting visible filter options; a consistent colour system and iconography across all three clients; WCAG AA contrast and content descriptions for screen readers.

6. Future Improvements. Real payment gateway (Stripe or Redsys); productive FCM push; AI-powered sustainability recommendations; OAuth2/SSO; and enhanced security (2FA, audit logs, Redis caching, CDN for images).

7. Conclusion. GaliciaWear demonstrates production-ready full-stack development across four platforms, with a coherent architecture, comprehensive security, automated testing, and a CI pipeline. It addresses a real market gap while covering the full spectrum of the DAM curriculum, from OOP and ORM to real-time communications and REST APIs.